

**MODUL PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBYEK**

**MODUL 5
Inheritance, Generalization, Superclass, dan Subclass**



Disusun oleh:

Nirwana Haidar Hari, S.Pd., M.Kom.

**POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
PROGRAM STUDI DI LUAR KAMPUS UTAMA SUMENEP
PENS PSDKU SUMENEP**

2026

MODUL PRAKTIKUM P5

Inheritance, Generalization, Superclass, dan Subclass

1. Tujuan Praktikum

Setelah menyelesaikan praktikum, mahasiswa mampu:

1. Menjelaskan konsep **inheritance**.
2. Mengidentifikasi class yang memiliki attribute atau behavior serupa.
3. Melakukan **generalization** untuk membentuk superclass.
4. Membuat subclass menggunakan extends.
5. Mengakses member superclass dari subclass.
6. Menggunakan super pada constructor.
7. Melakukan refactoring proyek P4 menggunakan inheritance.
8. Menghasilkan *project increment* melalui Agile Sprint.

2. Review P4

Pada P4, mahasiswa telah menghubungkan beberapa object menggunakan:

- association,
- aggregation,
- composition.

Contoh:

Anggota ----- Peminjaman ----- Buku

Seiring proyek berkembang, dapat muncul beberapa class dengan attribute dan method yang sama.

Contoh:

Mahasiswa

nama
alamat
nomorTelepon

Dosen

nama
alamat
nomorTelepon

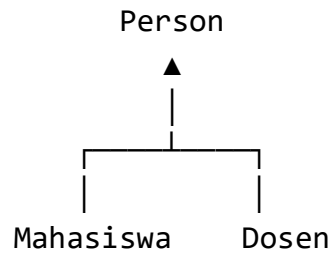
Terjadi duplikasi.

Pada P5, struktur tersebut diperbaiki dengan **generalization dan inheritance**.

3. Konsep Singkat

A. Inheritance

Inheritance memungkinkan sebuah class memperoleh attribute dan method dari class lain.



Person disebut:

Superclass

Sedangkan:

Mahasiswa

Dosen

disebut:

Subclass

B. Generalization

Generalization dilakukan dengan mencari karakteristik yang sama dari beberapa class.

Sebelum:

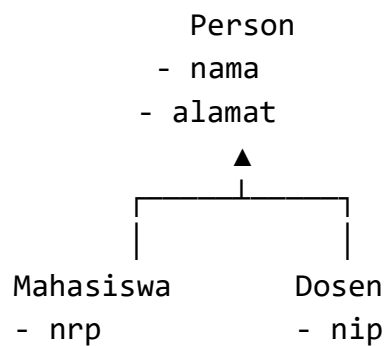
Mahasiswa

- nama
- alamat
- nrp

Dosen

- nama
- alamat
- nip

Setelah generalization:



Attribute umum dipindahkan ke superclass.

4. Implementasi extends

Superclass:

```
public class Person {  
  
    private String nama;  
    private String alamat;  
  
    public Person(  
        String nama,  
        String alamat  
    ) {  
  
        this.nama = nama;  
        this.alamat = alamat;  
    }  
  
    public String getNama() {  
        return nama;  
    }  
  
    public String getAlamat() {  
        return alamat;  
    }  
}
```

Subclass:

```
public class Mahasiswa  
    extends Person {  
  
    private String nrp;  
  
    public Mahasiswa(  
        String nama,  
        String alamat,  
        String nrp  
    ) {  
  
        super(  
            nama,  
            alamat
```

```

        );

        this.nrp = nrp;
    }

    public String getNrp() {
        return nrp;
    }
}

```

5. Keyword super

super digunakan untuk mengakses bagian superclass.

Contoh:

```

    super(
        nama,
        alamat
    );

```

berarti memanggil constructor superclass.

Alurnya:

```

new Mahasiswa(...)
    ↓
Constructor Mahasiswa
    ↓
super(...)
    ↓
Constructor Person

```

6. Constructor Chaining

Ketika subclass dibuat, constructor superclass dijalankan terlebih dahulu.

Contoh:

```

public class Kendaraan {

    public Kendaraan() {
        System.out.println(
            "Constructor Kendaraan"
        );
    }
}

```

Subclass:

```

public class Mobil

```

```

        extends Kendaraan {

        public Mobil() {
            System.out.println(
                "Constructor Mobil"
            );
        }
    }
}

```

Output:

```

Constructor Kendaraan
Constructor Mobil

```

7. Konsep is-a

Inheritance sebaiknya digunakan jika hubungan antarclass benar-benar:

is-a

Contoh benar:

```

Mobil is-a Kendaraan
Mahasiswa is-a Person
Kucing is-a Hewan

```

Contoh kurang tepat:

Mesin is-a Mobil

Hubungan tersebut lebih tepat:

Mobil has-a Mesin

yang biasanya menggunakan composition atau aggregation.

8. Kapan Menggunakan Inheritance?

Gunakan inheritance jika:

- terdapat hubungan is-a,
- beberapa class memiliki karakteristik umum,
- superclass benar-benar merepresentasikan konsep yang lebih umum,
- inheritance mengurangi duplikasi kode.

Jangan menggunakan inheritance hanya karena ingin menggunakan extends.

9. Agile Sprint – P5

Sprint Goal

Memperbaiki desain proyek dengan mengidentifikasi class yang memiliki karakteristik serupa dan melakukan generalization menggunakan inheritance.

Sprint Backlog

Contoh:

ID	Sprint Backlog	Status
SB-01	Mencari class yang memiliki duplikasi	To Do
SB-02	Menentukan superclass	To Do
SB-03	Menentukan subclass	To Do
SB-04	Memperbarui class diagram	To Do
SB-05	Implementasi extends	To Do
SB-06	Implementasi super	To Do
SB-07	Melakukan pengujian inheritance	To Do

Status:



10. Tugas Praktikum

Tugas P5 – Refactoring Proyek dengan Inheritance

Gunakan **project hasil P4**.

Mahasiswa tidak membuat project baru.

11. Bagian A – Audit Class

Analisis seluruh class project.

Cari minimal **2 class** yang:

- memiliki attribute serupa,
- memiliki behavior serupa,
- dapat digeneralisasi.

Contoh:

Class	Attribute Umum	Attribute Khusus
Mahasiswa	nama, alamat	nrp
Dosen	nama, alamat	nip

Kemudian tentukan:

Superclass:

Person

Subclass:

Mahasiswa

Dosen

12. Bagian B – Update Class Diagram

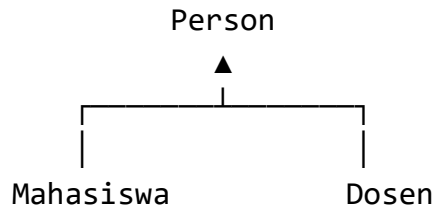
Perbarui class diagram.

Sebelum:

Mahasiswa

Dosen

Setelah:



Class diagram minimal menampilkan:

- superclass,
- subclass,
- generalization,
- attribute,
- method,
- relasi lain dari P4.

13. Bagian C – Implementasi Superclass

Buat minimal **1 superclass**.

Contoh:

```
public class Person {

    private String nama;
    private String alamat;

    public Person(
        String nama,
        String alamat
    ) {

        this.nama = nama;
        this.alamat = alamat;
    }

    public String getNama() {
        return nama;
    }

    public String getAlamat() {
```



```

        return alamat;
    }
}

```

Superclass harus berisi attribute atau behavior yang benar-benar umum.

14. Bagian D – Implementasi Subclass

Buat minimal **2 subclass**.

Contoh:

```

public class Dosen
    extends Person {

    private String nip;

    public Dosen(
        String nama,
        String alamat,
        String nip
    ) {

        super(
            nama,
            alamat
        );

        this.nip = nip;
    }

    public String getNip() {
        return nip;
    }
}

```

15. Bagian E – Implementasi super

Minimal gunakan:

```
super(...)
```

untuk memanggil constructor superclass.

Mahasiswa harus dapat menjelaskan:

- mengapa super() digunakan,
- constructor mana yang dipanggil,
- urutan eksekusi constructor.

16. Bagian F – Pengujian

Buat object dari minimal **2 subclass**.

Contoh:

```
public class Main {

    public static void main(
        String[] args
    ) {

        Mahasiswa mhs =
            new Mahasiswa(
                "Budi",
                "Surabaya",
                "3123001"
            );

        Dosen dosen =
            new Dosen(
                "Andi",
                "Surabaya",
                "198001"
            );

        System.out.println(
            mhs.getNama()
        );

        System.out.println(
            dosen.getNama()
        );
    }
}
```

Pastikan attribute superclass dapat digunakan oleh subclass melalui mekanisme yang sesuai.

17. Uji Sebelum dan Sesudah Refactoring

Mahasiswa harus menunjukkan perubahan desain.

Contoh:

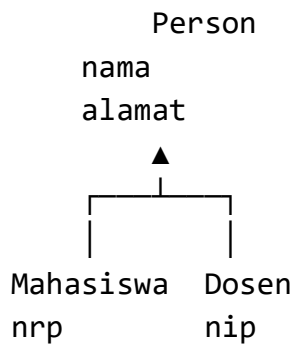
Sebelum

Mahasiswa

nama
alamat
nrp

Dosen
nama
alamat
nip

Sesudah



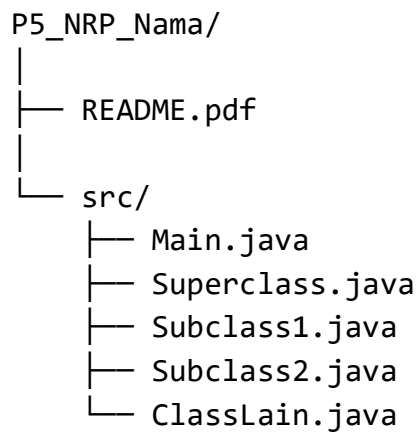
Tuliskan manfaat refactoring.

Contoh:

- mengurangi duplikasi,
- struktur class lebih jelas,
- maintenance lebih mudah.

18. Luaran Tugas

Kumpulkan:



README.pdf maksimal **3 halaman**.

Halaman 1

- Nama project
- Sprint Goal

- Sprint Backlog
- Audit class
- Kandidat superclass dan subclass

Halaman 2

- Class Diagram sebelum refactoring
- Class Diagram setelah refactoring
- Penjelasan hubungan is-a

Halaman 3

- Screenshot hasil running
- Bukti penggunaan extends
- Bukti penggunaan super
- Sprint Review
- Sprint Retrospective

19. Sprint Review

Item	Hasil
Kandidat inheritance ditemukan	...
Superclass berhasil dibuat	...
Minimal 2 subclass dibuat	...
extends diterapkan	...
super diterapkan	...
Class diagram diperbarui	...
Program berhasil dijalankan	...
Kendala	...

20. Sprint Retrospective

Jawab secara singkat.

What Went Well?

Apa bagian inheritance yang berhasil diterapkan?

What Went Wrong?

Apa kesulitan dalam menentukan superclass dan subclass?

Improvement

Apa yang perlu diperbaiki pada desain proyek?

Maksimal **3–5 kalimat**.

21. Definition of Done

P5 selesai jika:

- Menggunakan project P4.
- Melakukan audit terhadap class.

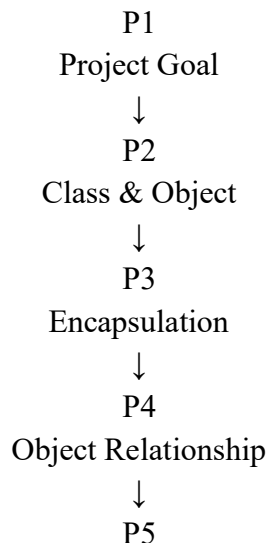
- Menemukan kandidat generalization yang relevan.
- Minimal 1 superclass dibuat.
- Minimal 2 subclass dibuat.
- Menggunakan extends.
- Menggunakan super.
- Constructor superclass dan subclass berjalan.
- Tidak terdapat duplikasi attribute umum yang seharusnya berada di superclass.
- Hubungan inheritance memenuhi prinsip is-a.
- Class Diagram diperbarui.
- Relasi dari P4 tetap dipertahankan.
- Program berhasil dikompilasi.
- Program berhasil dijalankan.
- Sprint Backlog diperbarui.
- Sprint Review tersedia.
- Sprint Retrospective tersedia.

22. Penilaian

Komponen	Bobot
Audit dan identifikasi generalization	15%
Class Diagram dan hubungan is-a	20%
Implementasi superclass & subclass	25%
Implementasi extends, super, dan constructor	20%
Pengujian program	10%
Agile Sprint & dokumentasi	10%
Total	100%

23. Target Sprint Berikutnya

Hasil P5 menjadi dasar untuk **P6 – Polymorphism**.



Inheritance
+
Generalization
↓
P6
Polymorphism
+
Overriding
+
Overloading
+
Dynamic Binding

Pada P6, subclass yang sudah dibuat tidak hanya mewarisi struktur superclass, tetapi mulai memiliki **behavior yang berbeda melalui method overriding dan polymorphism.**